Modular Python Cybersecurity Projects

NETWORK MONITORING • DEVICE DISCOVERY • OFFLINE LOGIN SIMULATION

> MAIN INSTRUCTION: Each team must select and complete EXACTLY TWO (2) projects from the three options in this workbook. Each selected project is marked out of 10; both together are marked out of 20. These are two complete, independently runnable modular Python applications, not two features of the same application.

Group name: _____ Class: _____

Group members: _____

Selected projects (tick TWO): ☐ A - Network Intrusion Detection ☐ B - Network Scanner ☐ C - Login Simulator

Submission date: _____ TOTAL: 20 MARKS (10 + 10)

1. Project overview and real-world relevance

Security teams rarely build professional tools as one giant Python script. Network sensors, discovery tools and authentication testing tools are easier to understand when input collection, analysis, reporting and file output are written as separate modules. This is called separation of concerns. Your task is to practice that structure while building two small security-focused applications.

The instructor assesses Python package design, imports, clear function boundaries, working demonstrations and documentation. Sophisticated detection methods, broad reconnaissance and real credential attacks are NOT required.

The three options — select any two

Choice	Main project	What the finished project demonstrates
A	Network Intrusion Detection System (NIDS)	Reads permitted lab packet traffic; summarizes IPs and patterns; raises alerts and writes an incident log.
B	Network Scanner & Device Discovery	Discovers reachable hosts inside a specifically authorized local subnet and reports IP / MAC when available.
C	Offline Brute-Force Login Simulator	Tests fictional passwords against an in-program dummy account and displays attempt limits and lockout behavior.

2. Safety, authorization and technical scope

• Only capture packets or discover devices on a network that the instructor or network owner has explicitly authorized for the activity. School/office/public networks are not automatically permitted.

• For network scanning, use a bounded instructor-approved private lab subnet. Do not scan public IP ranges, unrelated devices or networks outside that scope.

• For NIDS, prefer an instructor-provided PCAP capture when live capture requires elevated permissions, Npcap or a suitable network interface; live capture is an extension with instructor permission.

• For Option C, use fictional test credentials stored locally in the program. Do not connect to real websites, services, accounts or authentication endpoints.

• Avoid storing packet payloads, personal credentials or sensitive network identifiers in GitHub. Sample or anonymized logs should be used in the submitted repository.

3. Prerequisite knowledge and learning outcomes

Before starting, you should know

• Variables, strings, lists, dictionaries and basic comparisons.

• Functions: parameters, return values and calling functions from another file.

• for/while loops and simple if statements.

• Working with files using open(), read(), write(), with and basic paths.

• Modules, packages, __init__.py and import / from ... import.

• Very basic networking concepts: IP address, port, packet, device, MAC address and a subnet.

After completing your two projects, you should be able to

• Create two organized Python package projects and run them through main.py.

• Explain which module is responsible for gathering data, analyzing it and reporting results.

• Use a third-party library only where it helps your selected project.

• Explain your project outputs, test cases and ethical boundaries.

4. General library concepts — read before your chosen projects

A MODULE is one Python file containing reusable code. A PACKAGE is a folder grouping related modules; your assignment includes an __init__.py file to make this explicit. A standard-library module comes with Python (for example: time, logging, pathlib, ipaddress). A third-party package is installed separately (for example: Scapy). Your own modules (such as scanner.py or alerts.py) are custom modules written by your team.

2A. Exactly which network may we use?

> FOR OPTIONS A AND B: Use only an instructor-approved classroom lab network, or a personal phone hotspot/home network owned by your team with the owner's permission AND the instructor's approval of the exact testing scope. Never test the school network, public Wi-Fi, neighbors' devices or internet IPs without specific authorization.

Recommended setup for Option B — Network Scanner

A student-controlled phone hotspot is a practical starting point. The hotspot creates a small local Wi-Fi network; it does NOT authorize scanning the phone carrier's network or the wider internet.

Step	What your group should do
1. Create the lab network	One member enables a password-protected hotspot on their own phone. Alternatively use an instructor-provided lab router.
2. Connect devices	Connect the scanning laptop and 1–2 devices owned by group members or explicitly authorized for testing.
3. Identify local addressing	On the scanning laptop, inspect its Wi-Fi IPv4 address, subnet mask/prefix and default gateway. Do not assume any fixed hotspot range.
4. Record approved scope	Write the permitted private subnet or small list of test IPs in the README. Ask the instructor to approve that exact scope before running discovery.
5. Run conservatively	Test only this small local range, with short timeouts and no rapid repeated probes. Record active/reachable responses and MAC addresses only if they are available.
6. Explain limitations	Record any isolation, filtering or missing MAC data rather than inventing results.

Finding the subnet (do not copy example addresses blindly)

Windows: open Command Prompt and use ipconfig. macOS: open Network settings and inspect Wi-Fi TCP/IP details. Linux: inspect your Wi-Fi connection settings or use ip address. Locate the IPv4 address and subnet prefix/mask; confirm the gateway and obtain an instructor-approved small target range.

Example ONLY: if your laptop is 192.168.43.20 with a /24 mask, the local subnet might be 192.168.43.0/24. For this beginner assignment, prefer a few approved device IPs or a bounded /28-style test range. A different phone may use a completely different subnet.

> A device not responding does not prove it is offline. Hotspot client isolation, firewalls, operating-system restrictions and local-link limitations can prevent discovery. A device's MAC address may be "Not available," and this is a valid documented outcome.

Relevant documentation: Android hotspot setup — https://support.google.com/android/answer/9059108?hl=en

2B. NIDS testing: live traffic or offline PCAP?

> OPTION A ACCEPTED PATHS: (1) Analyze an instructor-approved offline PCAP file (recommended for beginners), OR (2) capture traffic generated by your own test laptop on a permitted interface. These are alternative input methods for the SAME NIDS project, not extra assignments.

Path 1 — Offline PCAP analysis (beginner-friendly default)

• Use a teacher-provided/safely generated .pcap file or a vetted non-sensitive sample packet capture.

• Load packets in nids/capture.py, then pass them to analyzer.py, alerts.py and incident_logger.py.

• Demonstrate the required three pattern checks: distinct destination ports, repeated attempts and unusually high traffic volume. Use a controlled dataset or synthetic packet records for tests if the PCAP lacks a pattern.

• Show terminal alerts and a timestamped incidents.log. Never upload real people's private packet payloads to GitHub.

Path 2 — Live capture of your own lab traffic (optional)

• With explicit network-owner and instructor permission, capture traffic to/from your own laptop while it is connected to an approved hotspot or isolated lab network.

• Capture only a short testing window. Do not assume the laptop can see all traffic between other hotspot-connected devices.

• Live capture may need system privileges or capture drivers. If the device cannot capture reliably, use Path 1 without losing marks.

What counts as a valid demonstration?

Project	Required evidence	Acceptable limitation
Option A — NIDS	PCAP or approved own-device traffic analyzed; IP/port summary; three simple alert demonstrations; saved incident log.	Offline processing is fully valid. Seeing only your own device traffic is expected on many networks.
Option B — Scanner	Approved hotspot or classroom subnet; attempts to identify reachable devices; IP/status table; saved report.	Missing MAC addresses or blocked client-to-client visibility must be described; do not fabricate device discoveries.

Safety and privacy — mandatory for both network options

• Before each test, document who owns the network, the approved IP scope or data source, and who gave permission.

• Do not attempt to bypass hotspot isolation, obtain other users' traffic, intercept credentials, or scan public addresses.

• Use sample/anonymized logs for GitHub; exclude raw private PCAP data, sensitive device identifiers and secrets.

Official sources: Scapy usage (PCAP/packet analysis): https://scapy.readthedocs.io/en/stable/usage.html

Wireshark capture permissions: https://wiki.wireshark.org/CaptureSetup

Wireshark sample captures: https://wiki.wireshark.org/SampleCaptures

2C. Assessment and GitHub hand-in rules

The original instructions still apply: every group selects exactly TWO of Options A, B and C. Each selected project is marked /10, so the combined score is /20. The third, unselected option carries no marks.

Fair grading on different hardware

> For Option A, offline PCAP analysis earns full eligibility for all 10 marks when the NIDS works and produces the required alerts and incident log. Live network capture is NOT mandatory. For Option B, students must attempt discovery only within an approved local environment; restrictions imposed by the hotspot, firewall or operating system should be explained in the README and demo, not bypassed.

Criterion	Marks per project	Evidence
Correct files, folders and package	2	main.py, __init__.py and designated module files.
Working local module imports	2	Functions from custom package modules are imported and called successfully.
Separation of concerns and data flow	2	Inputs, analysis and output move correctly between modules.
Basic functionality	2	Each selected option’s core features and acceptance checks work using safe test data.
Report and testing	1	Output log/report and clear test evidence.
README, GitHub and explanation	1	Readable instructions, proper repository submission and team demonstration.
TOTAL	10	Two completed projects = 20 marks.

GitHub submission — exactly what to hand in

• Push BOTH selected projects, each with its own main.py and package modules, to GitHub. They can be in a single repository under two named folders, or in two repositories.

• Each project must include README.md, dependency/install details, setup and run instructions, sample output/log, and the approved test scope/data source. Explain the limitation if a hotspot blocks discovery or live capture.

• Do not push secrets, private device lists or sensitive PCAP recordings. Use sanitized examples or explain how the teacher can obtain an approved lab file.

• Open the repository in a browser and verify all expected files are visible. Submit ONE GitHub homepage URL if using one repository, or TWO URLs if using two repositories.

> SUBMISSION TEMPLATE: Group name and members | Chosen options (e.g., A + B) | GitHub repository URL(s) | Approved test network/data source | One sentence explaining how to run EACH program.

GitHub documentation — add locally hosted code: https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github

Reminder: The source project descriptions, architecture blueprints and option-specific library references elsewhere in this workbook remain part of the assignment. These inserted pages clarify the testing environment and do not add a third project.

A good reading sequence: first understand what the library does; next find the two or three functions your task needs; then test a tiny example on safe sample data; only afterward connect it to main.py.

Python tutorial: Modules and packages — https://docs.python.org/3/tutorial/modules.html
Learn import statements, reusable functions, packages and __init__.py.

Python standard library index — https://docs.python.org/3/library/index.html
Learn what comes with Python; these modules usually do not need pip.

Python Packaging: installing packages — https://packaging.python.org/en/latest/tutorials/installing-packages/
Learn what pip does and how to install third-party tools.

Python files tutorial — https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files
Read files and write reports without external packages.

5. OPTION A — Network Intrusion Detection System (NIDS)

> MAIN PROJECT QUESTION — OPTION A: Build ONE modular Python network-monitoring application that reads authorized network packet data, identifies source and destination IP addresses, recognizes basic suspicious traffic patterns, displays alerts, and writes an incident log. Run it on an instructor-approved lab capture. Live capture on an authorized interface is an optional or instructor-approved mode.

5.1 What must the application do?

• Read a provided PCAP file of sample traffic; optionally capture live traffic on an approved lab interface.

• Extract and summarize available source IP addresses, destination IP addresses, and destination ports from observed packets.

• Identify three simple patterns: a source trying several destination ports (possible port scan), repeated connections to one destination, and an unusually high packet count within a short interval.

• Generate an alert containing a timestamp, alert type and relevant IP or port information.

• Write alerts to a human-readable incident log and display a brief summary in the terminal.

A detection alert means “review this traffic,” not proof that an attack occurred. Use simple, clearly documented thresholds chosen by your group; include the threshold values in your README.

5.2 Required project folder architecture

FOLDER STRUCTURE — CREATE THESE FILES

```
network-intrusion-detector/
main.py
requirements.txt
README.md
nids/
__init__.py
capture.py
analyzer.py
alerts.py
incident_logger.py
sample_data/
lab_traffic.pcap (instructor-provided or safely generated)
output/
incidents.log
```

5.3 What belongs in each file?

File	Main responsibility
nids/capture.py	Read offline PCAP packet records (and optionally approved live interface capture). Return packets or simplified packet records.
nids/analyzer.py	Extract IP/port fields and count repeated requests, distinct ports, and packet volumes. Return event/alert candidates.
nids/alerts.py	Turn detected patterns into understandable alerts with type, source, timestamp and reason.
nids/incident_logger.py	Append alerts to output/incidents.log; make the output readable and easy to inspect.
main.py	Import and coordinate capture → analysis → alert formatting → incident logging.

5.4 Acceptance tests / demonstration

☐ A normal sample capture can run without an application crash.

☐ A test capture or prepared synthetic packet list produces at least one alert for each of the three stated patterns.

☐ The incident log is created and contains time, alert type and context.

☐ You can explain why a repeated connection or high volume might be benign.

5.5 Learn your libraries FIRST — Option A

Scapy is a third-party Python packet-processing package. Read its usage guide to learn how packet records are loaded and how layers such as IP and TCP are inspected. Scapy can also capture traffic, but permissions and drivers can differ across operating systems. Use PCAP reading first if you are new.

Scapy usage and PCAP handling — https://scapy.readthedocs.io/en/stable/usage.html
Focus on rdpcap(), sniff(offline=...), packet layers and packet summaries.

Scapy installation guide — https://scapy.readthedocs.io/en/stable/installation.html
Scapy generally needs pip installation; some systems require packet capture drivers or elevated privileges for live traffic.

The Python logging module is included in the standard library. It provides basic methods for writing timestamped log entries to a file. The pathlib module helps construct file paths.

Python logging HOWTO — https://docs.python.org/3/howto/logging.html
Focus on basicConfig(), logger.info(), logger.warning() and file logging.

Python pathlib documentation — https://docs.python.org/3/library/pathlib.html
Focus on Path, mkdir(), exists() and file paths.

6. OPTION B — Network Scanner & Device Discovery

> MAIN PROJECT QUESTION — OPTION B: Build ONE modular Python tool that examines a SMALL, instructor-approved local IP address range; reports which devices respond; displays their IP addresses; and retrieves MAC addresses when the discovery method and local network make this possible. Present the results in a readable table and save a report.

6.1 What must the application do?

• Accept a bounded, instructor-approved private subnet or a small list of lab IP addresses; validate that it belongs to the approved scope.

• Try to discover reachable devices inside that permitted range, with a reasonable timeout and conservative request rate.

• Display each discovered IP address, a reachability status, and a MAC address if available.

• Show “Not available” when a MAC address cannot be collected; do not invent MAC addresses.

• Save a summary report (CSV or text) and show the number of responding devices.

Beginner scope: Limit the first demonstration to a small lab subnet, such as /28 or a shorter explicit list provided by the instructor. The exact range must be approved by the instructor. Do not add port scanning, service exploitation, stealth features or internet-wide discovery.

6.2 Required project folder architecture

FOLDER STRUCTURE — CREATE THESE FILES

```
network-device-discovery/
main.py
requirements.txt
README.md
discovery/
__init__.py
scope.py
scanner.py
formatter.py
reporter.py
output/
devices.csv
```

6.3 What belongs in each file?

File	Main responsibility
discovery/scope.py	Validate the permitted input subnet/IP list, enumerate only allowed addresses, and enforce the range limit.
discovery/scanner.py	Perform basic local discovery with a conservative timeout. Return results for reachable devices and MAC if available.
discovery/formatter.py	Organize IP, MAC, status and readable summary information for display.
discovery/reporter.py	Write the final results to output/devices.csv or a simple text report.
main.py	Import modules and coordinate scope validation → discovery → display → export.

6.4 Acceptance tests / demonstration

☐ Invalid or unapproved ranges are rejected before any network request is made.

☐ A small approved lab target list produces a readable table.

☐ Unreachable devices are not falsely labeled as active.

☐ Missing MAC addresses are reported accurately as unavailable.

☐ An output file is generated with the discovered information.

6.5 Learn your libraries FIRST — Option B

The built-in ipaddress module handles IPv4 addresses and subnets. Learn this before implementing discovery so your application can validate and restrict the scan scope. Scapy is an optional third-party library for local Ethernet/ARP discovery. ARP may reveal MAC addresses on the same local link, but it will not reveal MAC addresses for arbitrary remote networks.

Python ipaddress documentation — https://docs.python.org/3/library/ipaddress.html
Focus on ip_network(), hosts(), is_private and CIDR prefix notation.

Scapy usage documentation — https://scapy.readthedocs.io/en/stable/usage.html
Read the basic packet sending/receiving and ARP-related examples for authorized local discovery.

Scapy installation guide — https://scapy.readthedocs.io/en/stable/installation.html
Review prerequisites. Windows can require Npcap; administrative privileges may be required.

Python csv documentation — https://docs.python.org/3/library/csv.html
Focus on DictWriter, writerow() and writing a CSV report.

Python socket documentation (optional) — https://docs.python.org/3/library/socket.html
Learn basic addresses and socket timeouts; no advanced probing is required.

7. OPTION C — Offline Brute-Force Login Simulator

> MAIN PROJECT QUESTION — OPTION C: Build ONE modular Python classroom simulator that uses a fictional local account and a short, instructor-created list of test passwords. Show how repeated guesses are handled, then demonstrate why attempt limits and temporary lockouts are helpful. The simulator must never send requests to a real login service.

7.1 What must the application do?

• Create one fictional account with a deliberately invented practice password.

• Read a short local practice list (for example five fictional guesses) and test each guess using a simple local comparison.

• Show attempt number, whether the attempt matched and whether a limit has been reached.

• Stop after a configurable maximum number of attempts and display a lockout message.

• Produce a short summary report showing total attempts and final state; do not record real passwords.

The emphasis is on how controls like rate limits and lockouts change the outcome. Use only the in-program dummy target; do not automate attempts against websites, SSH, Wi-Fi routers or any real authentication service.

7.2 Required project folder architecture

FOLDER STRUCTURE — CREATE THESE FILES

```
offline-login-simulator/
main.py
README.md
simulator/
__init__.py
target.py
wordlist.py
attacker.py
reporter.py
data/
dummy_guesses.txt
output/
attempt_summary.txt
```

7.3 What belongs in each file?

File	Main responsibility
simulator/target.py	Define a dummy account and a simple function that compares a submitted value to the fictional secret.
simulator/wordlist.py	Read a short set of fictional passwords from a local sample file.
simulator/attacker.py	Coordinate the test attempts, count them, and enforce the maximum-attempt policy.
simulator/reporter.py	Display and save the summary of attempts and the final account state.
main.py	Import and run target → guess list → policy-controlled attempts → report.

7.4 Acceptance tests / demonstration

☐ A fictional matching guess is recognized only when it occurs before the attempt limit.

☐ Wrong guesses are counted correctly.

☐ A low attempt limit causes the test to stop and display the lockout state.

☐ Output is clearly marked as fictional/offline practice data.

☐ The simulator never interacts with a live authentication system.

7.5 Learn your libraries FIRST — Option C

Option C does not require a third-party package. Python’s built-in lists, loops, strings, file handling and optional time module are enough. Students should understand those before connecting the simulator modules.

Python control flow tutorial — https://docs.python.org/3/tutorial/controlflow.html
Focus on for, while, break and defining functions.

Python input/output and file handling — https://docs.python.org/3/tutorial/inputoutput.html
Focus on reading the dummy wordlist and writing a report.

Python time documentation (optional) — https://docs.python.org/3/library/time.html
Focus on time.sleep() only for a visual classroom demonstration of delays.

8. How to organise the group sprint

Step 1 — Make your two choices

As a group, select exactly TWO options. Write each project title in your README. Be realistic about your time, lab equipment and permissions. For networking options, obtain scope approval from the instructor first.

Step 2 — Read the required library resources

Everyone reads the general module and package links in Section 4. Then read only the library pages for your two selected options. Each member should explain one function they expect to use and what it returns.

Step 3 — Build each project as a separate Python package

Create the required folders and files from each option blueprint. Assign module responsibilities, keep main.py as the coordinator, and test functions individually before connecting them with imports.

Step 4 — Run acceptance tests and prepare evidence

For EACH project, show one successful run plus at least one example of an error or alert condition appropriate to the task. Confirm any output report/log exists and is safe to share.

Step 5 — Prepare a short demonstration

Each team member must explain one file or function, why that code belongs there, and how its output reaches the next module. Aim for a brief demonstration of both selected projects.

9. GitHub submission — mandatory and explicit

> SUBMISSION METHOD: Push BOTH completed projects to GitHub and submit the GitHub repository link(s) to your instructor. A screenshot or a local ZIP file alone does not count as the required submission. Submit ONE link if both projects are stored in one repository; submit TWO links if your team uses a separate repository for each project.

Required GitHub repository contents

• Complete source code and correct folder/package architecture for BOTH selected projects.

• A README.md for each project identifying the option, team members, installation steps, approved environment, run instructions and example output.

• requirements.txt listing any third-party library (for example Scapy). For projects using only the standard library, state that no third-party dependency is required.

• Safe sample/test data and sanitized reports or screenshots. Never push raw sensitive capture data, real credentials, tokens, personal information or private network inventories.

• A root README if using a combined repository, with a link to each project subfolder.

How to submit

• Create a GitHub repository (or two repositories) and upload/push the complete tested project(s).

• Open the repository on GitHub and confirm every required folder and main.py is visible.

• Read the README from the browser and verify the setup instructions are sufficient.

• Copy the repository homepage URL (example: https://github.com/team-name/cyber-python-projects).

• Send the instructor: group name, members, selected TWO options, GitHub URL(s), and one sentence explaining how to run each project.

Official GitHub guide: adding local code — https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github
Official instructions for creating a repository and pushing existing source code.

Suggested submission format

Group: _____ Members: _____

Selected projects: Option ___ and Option ___

Repository URL 1: _____

Repository URL 2 (if separate): _____

Run instructions: See README.md in each selected project folder.

10. Marking scheme — EACH project is /10

Criterion	Per project	Evidence for full marks
Required modular folder structure	2	Correct package folder, __init__.py, main.py and specified modules.
Correct module imports	2	main.py imports the custom modules successfully; no missing imports.
Separation of concerns and data flow	2	Modules have distinct responsibilities; results move correctly through functions.
Core required features work	2	Main features and the basic project acceptance tests succeed.
Report/output and tests	1	Clear terminal output and a useful saved report/log or evidence.
README, GitHub submission and explanation	1	Repo is complete, readable and one member can explain the data flow.
TOTAL PER PROJECT	10	10 marks for each selected project.

Combined group score

Selected project 1	Selected project 2	TOTAL
_____ / 10	_____ / 10	_____ / 20

There are no marks for the unselected third option. Sophisticated attack logic, high scan volume, real target probing or unauthorized packet collection will not earn extra marks.

11. Beginner troubleshooting & final checklist

Problem	What to check
ModuleNotFoundError	Check the package directory name, the __init__.py file, your import statements and whether you run from the project root.
Import succeeds, but the app does nothing	Check whether main.py actually calls the function and handles its return value.
Permission or driver error for packet capture	Use authorized offline PCAP mode and consult Scapy installation requirements; ask your instructor before changing privileges.
No devices found	Confirm lab authorization, correct subnet, interface choice and whether the hosts actually respond. Silence is not proof a device is offline.
Missing MAC addresses	MAC discovery is normally local-link dependent. Report “Not available” instead of inventing a value.
Output file missing	Confirm output folder exists and the report module is called with the correct path.

☐ We selected exactly two options, not three.

☐ Both projects have independently runnable main.py files.

☐ We read the library/function references for both of our chosen options.

☐ Our network targets or packet data are authorized (if applicable).

☐ Each project satisfies its own acceptance tests.

☐ Our GitHub repository/repositories contain the necessary code and documentation.

☐ We have removed secrets and sensitive or unnecessary packet data.

☐ We have the repository link(s) ready to submit.

END OF WORKBOOK • CYBERSECURITY TRACK • TWO PROJECTS • 20 MARKS